DS n° 04 – Le labyrinthe

- Faire tous les exercices dans un fichier NomPrenom.py à sauvegarder,
- mettre en commentaire l'exercice et la question traités (ex : # Exercice 1),
- ne pas oublier pas de commenter ce qui est fait dans votre code (ex : # Je crée une fonction pour calculer la racine d'un nombre),
- il est possible de demander un déblocage pour une question, mais celle-ci sera notée 0,
- il faut vérifier avant de partir que le code peut s'exécuter et qu'il affiche les résultats que vous attendez. Les lignes de code qui doivent s'exécuter sont décommentées.

Contexte et cadre général

On cherche à résoudre un labyrinthe, figure 1, représenté sous forme d'une **grille de** 14×20 **cases**.

Chaque case est soit un **mur** (en noir, valeur `1`), soit une **case libre** (en blanc, valeur `0`).

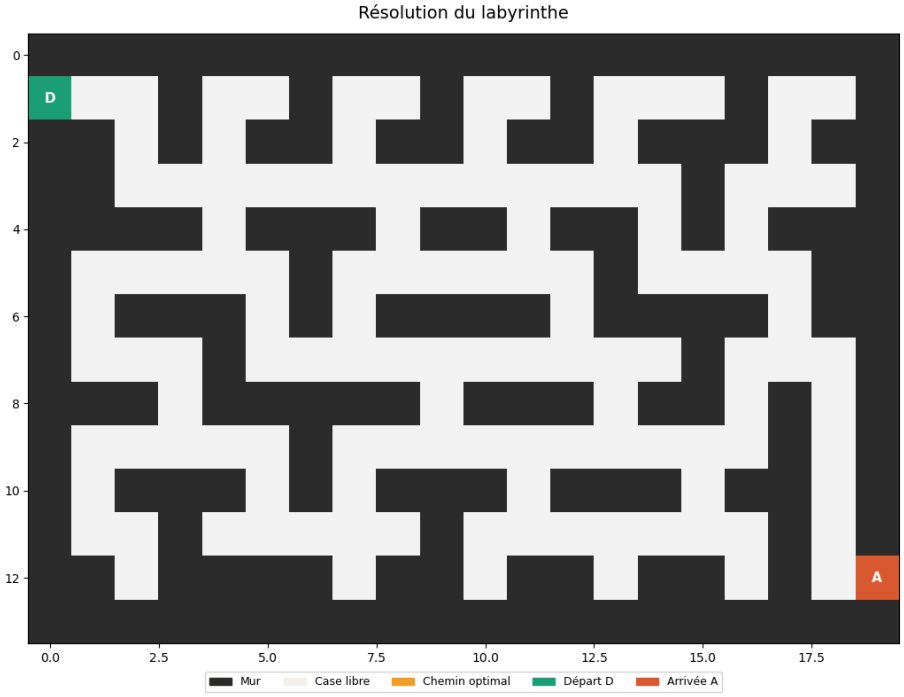


FIGURE 1 – Labyrinthe à résoudre

- Le **départ** D est situé en `(ligne=1, col=0)`.
- L'**arrivée** A est en `(ligne=12, col=19)`.

La grille est stockée dans un fichier CSV (`grille.csv`) où les cellules sont séparées par des `;` et chaque ligne correspond à une rangée du labyrinthe.

Le squelette du programme (imports, constantes, fonction d'affichage `afficher`) vous est fourni dans un fichier `script_candidat.py`. Vous devez le copier/coller à l'intérieur de votre fichier `NomPrenom.py`.

1 Analyse du labyrinthe

1.1 Lecture du fichier CSV

Question 1 **Écrire** le code permettant de lire le fichier `grille.csv` et de construire la variable `grille`, une liste de listes d'entiers (0 ou 1) représentant le labyrinthe. **Afficher** la première ligne (index=0) de `grille`.

1.2 Affichage initial du labyrinthe

Pour faire la question suivante copier/coller le contenu du fichier `script_candidat.py` dans votre script.

Question 2 **Appeler** la fonction `afficher()` pour afficher le labyrinthe.

1.3 Recherche des cases libres

Question 3 **Écrire** le code qui construit la liste `cases_libres` contenant toutes les cases de coordonnées $[i, j]$ telles que `grille[i][j] == 0`. **Afficher** la deuxième case (index=1) de `cases_libres`.

1.4 Nommage des cases

Question 4 **Écrire** la fonction `nom_case(case)` qui, pour une case donnée sous forme de liste `[i, j]`, renvoie la chaîne de caractères `'[i,j]'`. **Afficher** `nom_case([1, 2])` qui doit retourner `'[1,2]'`.

2 Parcours de graphe

2.1 Case suivante dans une direction

On définit les quatre directions de déplacement :

```
directions = [[0, 1], [0, -1], [1, 0], [-1, 0]]
```

Question 5 **Écrire** la fonction `prochaine_case(case, direction)` qui renvoie la case obtenue en appliquant le déplacement `direction` depuis `case`. **Afficher** `prochaine_case([0, 1], directions[2])` qui doit retourner `[1, 1]`.

2.2 Construction du graphe

Question 6 **Construire** le dictionnaire `graphe` tel que :

- Chaque **clé** est le nom (chaîne de caractère) d'une case libre,
- La **valeur** associée est la liste des noms des cases libres adjacentes (haut, bas, gauche, droite) qui sont dans les limites de la grille.

Attention aux bords de la grille !
Afficher `graphe['[1,1]']` qui doit renvoyer `['[1,2]', '[1,0]']`.

3 Résolution du labyrinthe

3.1 Recherche de tous les chemins

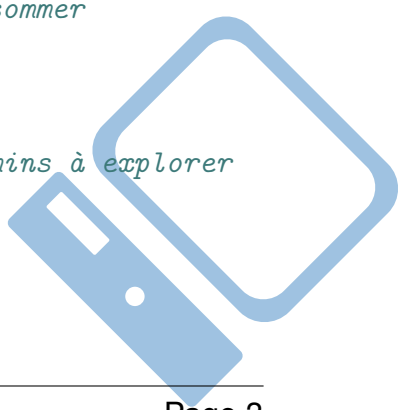
La fonction `tous_les_chemins(graphe, depart, arrivee)` permet de parcourir tous les chemins possible d'une case `depart` à une case `arrivee`.

Question 7 **Recopier** et compléter le code afin que la fonction renvoie les chemins attendus. Il n'est pas nécessaire de recopier les commentaires `#`.

```
def tous_les_chemins(graph, depart, arrivee):

    chemins_a_explorer = [[depart]]
    resultats = []

    while len(chemins_a_explorer)>0:
        # Récupérer le premier chemin des chemins a explorer
        # Le supprimer des chemins a explorer
        chemin = chemins_a_explorer.pop(0)
        # Partir du dernier sommet de ce chemin
        ..................
        # Si on atteint l'arrivée
        if sommet == arrivee:
            # Stocker le chemin dans la liste resultats
            ......................
        else:
            # Pour chaque voisin de ce sommet
            for voisin in graph[sommet]:
                # On regarde si on est pas déjà passé par ce sommer
                if voisin not in chemin:
                    # On ajoute ce voisin à ce chemin
                    ................................
                    # On intègre ce chemin à la liste des chemins à explorer
                    ................................
    return resultats
```



3.2 Chemin le plus court

Question 8 En utilisant le résultat de la fonction `tous_les_chemins`, **écrire** le code qui trouve le chemin de longueur minimale. **Appeler** la fonction `afficher(grille,chemin_plus_court)` qui affiche alors l'image du graphe avec le chemin le plus court en jaune.

FIN

